

Learning LLMs: The 20% That Matters

(The Foundation Everything Else Sits On)

1 The Whole Thing in One Sentence

A large language model is a **next-token predictor**: given some text, it outputs a probability distribution over what the next token should be, picks one, appends it, and repeats. Everything else — chatting, coding, reasoning, tool use — is built on top of that single capability. This document is the foundation underneath every other tool in the series; you have been building systems *around* this object, and now we open it up.

2 Tokens (The Unit of Everything)

LLMs don't see words or letters; they see **tokens** — chunks of text (a word, part of a word, or a symbol) mapped to integer IDs. “Tokenization” is the splitting step (the same one behind the Hugging Face document).

```
1 "unbelievable" -> ["un", "believ", "able"] -> [1234, 5678, 910]
```

Why it matters: the model's cost, context limit, and even some quirks (it struggles to count letters or do exact arithmetic) come from the fact that it operates on tokens, not characters or numbers. **You pay per token, in and out.**

3 Embeddings (Meaning as Geometry)

Each token ID is looked up in a big table and turned into a **vector** (a list of numbers) called an embedding. The model learns to place tokens with related meanings near each other in this space. This is the same idea that powers retrieval in RAG — “meaning becomes geometry.” Everything the model does internally is arithmetic on these vectors.

4 The Transformer (The Architecture)

Modern LLMs are **Transformers**. The input is the sequence of token embeddings; the output is a prediction for the next token. A Transformer is a stack of identical layers, each containing two main parts: an **attention** block and a small **feed-forward** network. Stacking many such layers (dozens to over a hundred) is what gives the model its depth and capability.

5 Attention (The Key Mechanism)

Attention is the core innovation. For each token, the model asks: *which other tokens in the sequence are relevant to me, and how much?* It then mixes in information from those tokens weighted by relevance. In “the animal didn't cross the street because *it* was tired,” attention is how the model

links *it* to *animal* rather than *street*. Repeated across many layers and many parallel “heads,” this lets the model build a rich, context-aware representation of every token. You don’t need the math; you need the intuition: **attention lets every token gather context from the whole sequence**.

6 How Text Is Generated (Autoregressive Loop)

Generation is one token at a time:

```
1 prompt -> model -> distribution over next token -> pick one -> append
2       -> feed the longer sequence back in -> next token -> ... -> stop
```

This is why responses stream in word by word, and why longer outputs cost proportionally more — each new token requires another pass. The model is “autoregressive”: its own output becomes its next input.

7 Decoding / Sampling (Controlling the Pick)

Once the model gives a distribution, how you choose the token shapes the output (the same knobs as the API and vLLM documents):

- **Greedy / temperature 0** — always take the most likely token. Deterministic, can be repetitive.
- **Temperature** — flattens or sharpens the distribution; higher = more random/creative.
- **Top-p (nucleus)** — sample only from the smallest set of tokens covering probability p .

The model is fixed; sampling decides how adventurously you read from it.

8 The Context Window (Working Memory)

The context window is the maximum number of tokens the model can attend to at once — your prompt plus its generation so far. It is the model’s *only* working memory: there is no hidden notepad. Two consequences:

- Anything not in the context effectively doesn’t exist for the model (hence RAG, which *puts* relevant facts into context).
- Attention cost grows roughly **quadratically** with length, so long context is expensive. Much engineering goes into extending effective context (today commonly 128k tokens and beyond) efficiently.

9 Scale and Parameters

“Parameters” are the learned numbers (weights) in the model; “7B” means 7 billion of them. Broadly, more parameters + more data + more compute yields more capability — the **scaling laws**. These laws still drive the field, but their exact shape shifts when the architecture or data pipeline changes, which is exactly what the next idea does.

10 Mixture of Experts (MoE) — The Modern Architecture

Activating every parameter for every token is wasteful. **MoE**, now standard in state-of-the-art systems, uses **conditional computation**: each token is **routed** to a small subset of specialized sub-networks (“experts”) via a top- k router, so a model can have a huge *total* parameter count while only a fraction is *active* per token.

- This is why you see names like “235B-A22B”: 235B total parameters, but only ~ 22 B **active** per token.
- Benefit: trillion-parameter capacity at reasonable per-token compute.
- Cost: complex training (load balancing, routing) and high memory to *hold* all experts in VRAM for serving.

11 How a Model Gets Its Behavior: The Training Recipe

This is the most important section for understanding model behavior. Modern LLMs are built in **multi-stage pipelines**:

1. **Pre-training** (self-supervised) — predict the next token over enormous amounts of internet-scale text. This is where the model learns language, facts, and reasoning patterns. It is by far the most expensive stage. The result is a “base model” that completes text but doesn’t follow instructions well.
2. **Supervised fine-tuning (SFT)** — train on curated instruction→response examples so the model learns to *follow instructions* and adopt a helpful format (the same SFT idea from the fine-tuning document, but done at scale by the model maker).
3. **Alignment** — preference-based training (RLHF and successors like DPO) shapes the model toward responses humans prefer: helpful, honest, safe.

Key insight: the base model knows the most; the later stages teach it how to *behave*. “The model won’t follow my format” and “the model knows a fact but won’t use it” are different problems precisely because knowledge and behavior come from different stages.

12 In-Context Learning and Prompting

A trained LLM can learn a task *from the prompt itself*, with no weight changes — give it a few examples (“few-shot”) and it generalizes. This is **in-context learning**, and it is why prompt engineering works at all: you are not changing the model, you are steering a fixed model by conditioning it on better context. It is also the cheapest lever (recall the build-decision ladder: prompt first, then RAG, then fine-tune).

13 Reasoning Models and Test-Time Compute

The major recent shift: models that “**think**” before answering. Instead of replying immediately, a reasoning model is trained to produce a long internal **chain-of-thought** — working through intermediate steps — before the final answer. Spending **more compute at inference time** (more thinking tokens) measurably improves results on hard tasks like math and coding. This

“test-time compute” is a new scaling dimension alongside model size: a smaller model that thinks longer can beat a bigger one that answers instantly. The trade-off is latency and cost, so you reserve heavy reasoning for genuinely hard problems.

14 The KV Cache (Why Serving Has Two Phases)

As the model generates, it caches the attention keys/values for tokens it has already processed so it doesn’t recompute them each step. This **KV cache** is the memory that grows with sequence length, and managing it efficiently (PagedAttention) is exactly what makes vLLM fast. Generation has two phases: *prefill* (process the whole prompt at once) and *decode* (generate one token at a time using the cache).

15 What LLMs Cannot Do (Be Honest About This)

- **No inherent notion of truth** — they predict plausible tokens, so they can state falsehoods fluently (hallucination). Grounding (RAG) and verification mitigate this.
- **Frozen knowledge** — a base model only knows its training data up to a cutoff; it has no live access to the world without tools/retrieval.
- **No persistent memory** — between calls it remembers nothing; “memory” is something the system around it provides via context.
- **Token-level blind spots** — counting characters, exact arithmetic, and spelling tricks are hard because it sees tokens, not characters or numbers.
- **Confidently wrong** — fluency is not accuracy; calibration is imperfect.

Understanding these is what makes the rest of the stack (RAG, tools, guardrails, evals) make sense — each one compensates for a specific limitation here.

16 The Mental Model in One Line

An LLM is a Transformer trained to predict the next token; pre-training gives it knowledge, fine-tuning and alignment give it behavior, the context window is its only memory, and everything you build around it exists to supply context, control sampling, ground it in truth, and let it act.

17 Key Takeaways

1. It predicts tokens; it does not “look things up” or “know” it is right.
2. Attention lets each token gather context; depth (many layers) builds capability.
3. Knowledge comes from pre-training; behavior comes from SFT + alignment.
4. The context window is the working memory — finite and quadratically costly.
5. MoE scales total size cheaply; reasoning models scale *thinking* at inference.
6. Its limitations are precisely why RAG, tools, guardrails, and evals exist.

18 Practice Task (Mostly Conceptual)

1. Use a tokenizer (from the Hugging Face document) to tokenize a sentence; count the tokens and see where words split.
2. Generate the same prompt at temperature 0 and temperature 1; observe determinism vs variety.
3. Take a question the model gets wrong from lack of knowledge, then supply the fact in the prompt (in-context) and watch it succeed — you just did manual RAG.
4. Ask a hard math/logic question two ways: “answer immediately” vs “think step by step first.” Compare accuracy — you just observed test-time compute.
5. Write one sentence each on which limitation in this document is addressed by: RAG, tools, guardrails, and evals.